

Usually, creating a website requires a huge time investment. You have to plan for the financial cost of developing, maintaining, and hosting it. If you're developing it yourself, you need to have significant technical knowledge. This often discourages a lot of people from building them. However, not all websites require some complicated architecture or costly resources to run. Sites like blogs, portfolios, company websites, magazines, documentation sites, etc. can be made cheaply and fairly hassle-free using static site generators.

Static site generators

A static site generator or SSG is a build tool that produces static sites. These sites have pages that are built beforehand and do not change often between user visits. An SSG usually takes templates and adds supplied data to them to create the HTML pages. The pre-rendered pages are then hosted where users can access them. This is unlike dynamic sites where a page is built when a user requests it.

SSGs simplify site development. In some cases, creating a site with an SSG is achieved in only a few steps and a very short period. SSGs are great tools for new developers and those with limited experience who'd like to make sites without much hassle. Many SSGs are resource-rich and provide a wide range of site themes, common utilities like comments and hosting integration, etc. Some other benefits of SSGs are tied to the static sites they produce. Static sites involve a lot fewer moving parts and are easier to build, secure, and maintain. They are fast since pages are already pre-rendered and are served as is. Scaling static sites is easier compared to other sites with more complex infrastructures.

HUGO

Hugo is an SSG and framework written in Go. It uses Go templates for its layouts. A key advantage that sets it apart from other SSGs is how fast it builds pages. It provides amazing content management, is available on multiple OS platforms, supports taxonomies, offers tables of contents generation, supports pretty URLs, and can be hosted virtually anywhere. In addition to these features, it provides live reload during development, supports different content type options, and has word-count as well as minutes-to-read functionalities. These are just a sampling of the many features Hugo has to offer.

Installation

Hugo is cross-platform. There are multiple ways you can install it. These include using package managers, from source code, or with Docker. However, the easiest way to get it installed is using binaries. Currently, Hugo has pre-built binaries for macOS, Linux, Windows, FreeBSD, and, OpenBSD. You can download them from Hugo's Github repository here. Be sure to install them on your path.

Once the installation is complete, run `hugo --help` on your terminal to make sure that the installation was complete and that it works. These are the first few lines you should see.

```
$ hugo --help
hugo is the main command, used to build your Hugo site.
Hugo is a Fast and Flexible Static Site Generator built with love by
spf13 and friends in Go.
```

Generating a Site

To create a new site, select a location for the site source code. Pick a site name and run the following command on your terminal in the location you picked.

```
$ hugo new site <site-name>
Congratulations! Your new Hugo site is created in <site-path>.
```

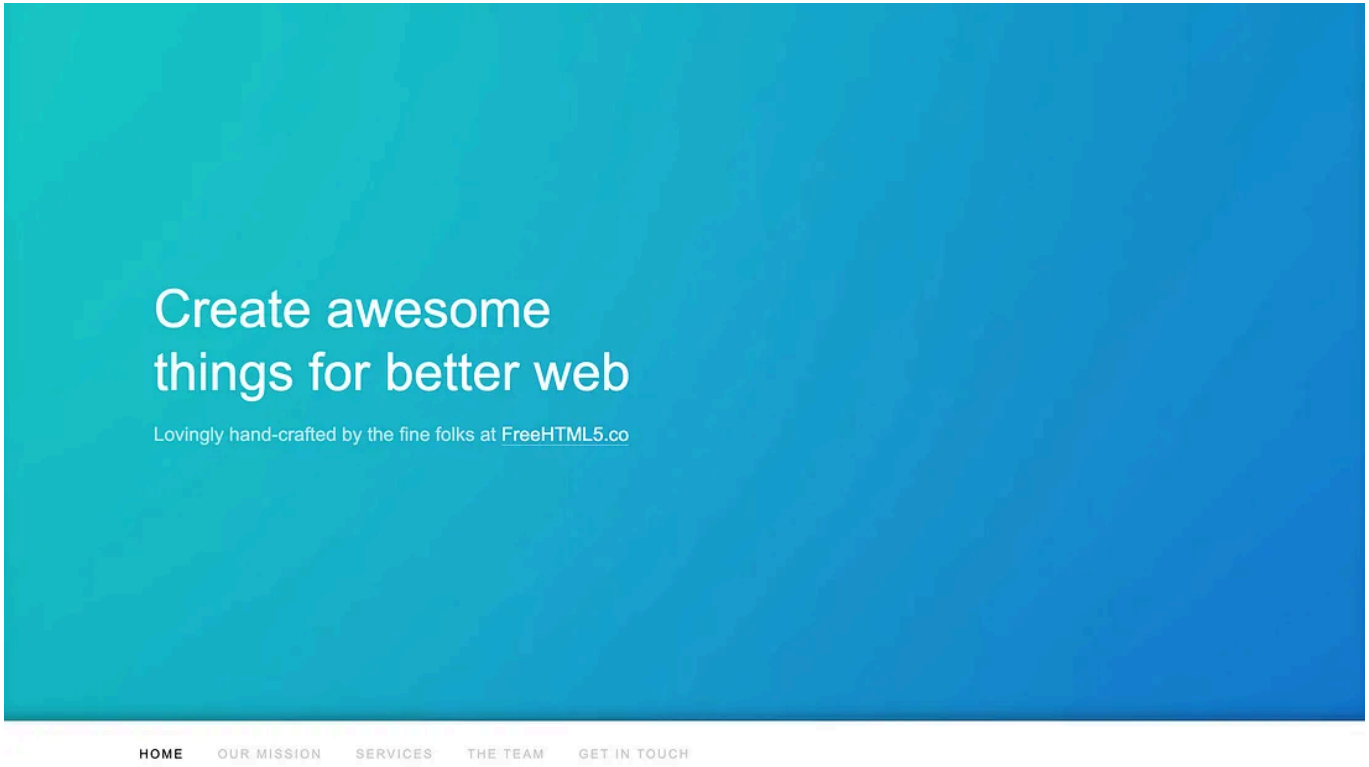
This command will generate a skeleton for your site. Most of the folders it generates are empty apart from `archetypes`. You may optionally choose to add a theme and make slight config changes instead of building the site from scratch. You can find a range of themes at theme.hugo.io. Adding them is usually the same for most themes. Although, some may have slight variations. To illustrate what adding a theme is like let's take the example of this startup website theme. Change directories to the site folder, initialize a git repository, and add the theme as a submodule.

```
$ cd <site-name>
$ git init
$ git submodule add https://github.com/SteveLane/hugo-icon.git
themes/hugo-icon
```

To run the site locally, with the Hugo server, the theme, and some example configuration, run:

```
$ hugo server -t hugo-icon --config themes/hugo-
icon/exampleSite/config.toml
```

You can visit the site at localhost:1313 on your browser and you'll see this.



Site Structure

A typical Hugo site would have this structure:

```
├── archetypes
├── config
├── content
├── data
├── layouts
├── resources
```

```
├── static
└── themes
```

Archetypes

Archetypes serve as templates for content files. For example, the default archetype that is created when a site is generated looks like this:

```
---
title: "{{ replace .Name "-" " " | title }}"
date: {{ .Date }}
draft: true
---
```

Any new content created using the `hugo new` command contains this as its front matter. Running `hugo new posts/hello.md` will create the following file in `contents/posts`:

```
---
title: "Hello"
date: 2021-05-20T11:03:02
draft: true
---
```

You can create new archetypes with custom front matter and structure. When you create new content, an archetype for it is deduced from the path if not explicitly set using the `--kind` flag. For example, if a profile archetype exists at `archetypes/profile.md`, you can create a new profile in the content folder using `hugo new profile/lisa-jane.md`.

Assets

Assets are files that require processing before they can be served. Hugo pipes are the functions that perform asset processing. For example, let's say there exists a SASS file. Its path is `assets/sass/form.sass`. To convert the SASS to CSS, you will use the `resources.ToCSS` pipe.

```
{{ $formSass := resources.Get "sass/form.sass" }}
{{ $formStyling := $formSass | resources.ToCSS }}
```

Hugo pipes perform minification, bundling, JS processing among other functions.

Config

By default, Hugo site configuration is stored in the `config.toml` file at the site root. You can configure any number of things like cache, URL, site titles, live reload, content directories, archetype directories, languages, builds, the dev server, etc. You may choose to have multiple configuration files for better organization. In this case, these will be stored in the `config` folder. The config can also be specified using the `--config` flag with a Hugo command. Files used with this flag take precedence over other configs. Here is an example of the config file for the startup site created above:

```
baseURL = "http://a-startup.com/"
languageCode = "en-us"
title = "My Startup Site"
```

Content

This is where the actual content of a site lives. In a blog, for example, the content will be all the posts, images, documents, and other resources. Hugo supports a variety of content types including HTML and markdown. Typically content instances contain front matter which is the metadata of the instance. In `posts/hello.md`, the front matter is the title, date, and draft status. You create a content instance using the `hugo new` command.

Data

This folder contains your site's extra data that's meant to augment the content. They can take the form of JSON, TOML, CSV, or YAML files. To get data from these files, you have to reference the `.Site.Data` variable. Let's take the example of a coffee shop site that has its menu in a `data/menus/beverages.json` file.

```
{
    "beverages": [
        { "name": "Americano", "price": "$2.50" },
        { "name": "Cappuccino", "price": "$3.75" },
        { "name": "Espresso", "price": "$3.00" },
        { "name": "Macchiato", "price": "$4.00" }
    ]
}
```

You can use this data in a template as follows:

```
{{ range $.Site.Data.menus.beverages }}
   {{ .name }}: {{ .price }}
{{ end }}
```

Although the `data` folder is meant for static data files, it is still possible to fetch JSON and CSV data from external sources such as CMSs like Strapi or other APIs. You could use functions like `getJSON` and `getCSV` for these purposes. You can find out more about how to access external data here.

Layouts

These are the templates of the site. Hugo templates use the `[text/template](https://golang.org/pkg/text/template/)` and `[html/template](https://golang.org/pkg/html/template/)` packages in Go. They are written in HTML but variables and functions can be used inside nested curly braces `{{ }}` within them. Using logic and referencing external templates inside them is possible. You can find a great primer on this template language at this link.

Resources

The resource folder acts as a cache. The files generated as a result of asset processing as mentioned above are stored here.

Static

Images, CSS, Javascript, and other static content are placed in this folder.

Themes

Themes added to a site are put in this folder. In the example site we generated above, we placed its theme here.

Functions

Within templates, you can use built-in Go and Hugo functions and tools. For example in the `archetypes/default.md`, you may have noticed this:

```
title: "{{ replace .Name "-" " " | title }}"
```

`replace` is a function that replaces part of a string with something else. In this case, the file name is the input, and all the `-` characters are removed then replaced with a single space. `title` is also a function that converts a string to title case. These are just a few examples. There are many more listed in Hugo's function reference.

Variables

Hugo makes inbuilt and config-defined values accessible to templates. These include variables that hold information about the site, pages, taxonomies,

content, menus, and other files. For example, the global `[.Site](http://(https://gohugo.io/variables/site/#site-variables-list)` variable holds site-related values like languages, sections, the site title, the base URL, etc.

Pipes

As mentioned earlier, Hugo pipes are functions that transform site assets. Some common uses of pipes include converting SCSS/SASS into CSS, resource minification, resource bundling, fingerprinting, and a host of other functions.

Conclusion

Hugo is an excellent static site generator that is fast and feature-rich. You can do so much with it and it offers a great deal of flexibility when working with site content. What we've covered here is just the tip of the iceberg. You can read about more interesting ways to use it in the Hugo documentation.

> This article is a guest post by **Zara Cooper**. She wrote this blog post through the Write for the Community program.